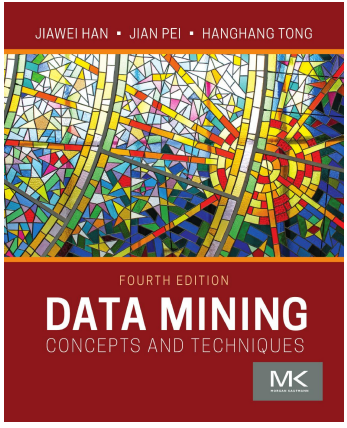
Emergent Digital Technologies

Artificial Neural Networks (ANN)

Jiawei Han & Jian Pei & Hanghang Tong

5 de noviembre de 2026

Data Mining: Concepts and Techniques



Content has been extracted from *Data Mining: Concepts and Techniques*, by Jiawei Han & Jian Pei & Hanghang Tong. Morgan Kaufmann. 2022. Visit https://shop.elsevier.com/books/data-mining/han/978-0-12-811760-6.

Introduction to Artificial Neural Networks

- Artificial Neural Networks (ANNs) are a family of techniques based on biological neural systems.
- A neural network is a set of connected input-output units where each connection has an associated weight.
- During learning, the network adjusts these weights to predict correct target values for input data.
- Applications include computer vision, natural language processing, and machine translation.

Introduction to Artificial Neural Networks (Cont.)

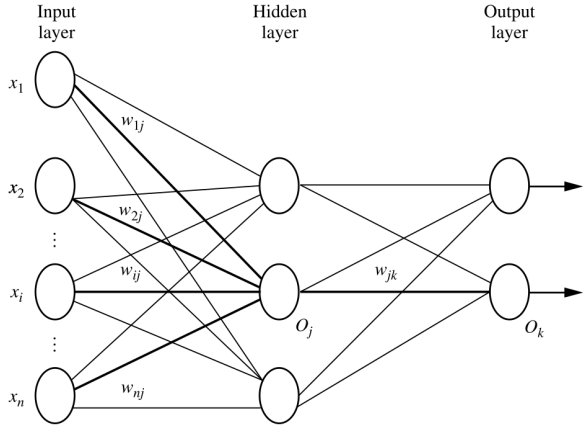


FIGURE 10.1

Multilayer feed-forward neural network.

What is a Unit?

- The basic building block of an ANN is the **unit**.
- Historically, basic classifiers like the perceptron and logistic regression are viewed as individual units.
- A unit is a mathematical function that:
  1. Takes a linear weighted sum of the inputs.
  2. Passes that sum through an activation function $f(\cdot)$.

Mathematical Representation of a Unit

- Given a data tuple $X = (x_1, x_2, \ldots, x_n)$:
- **Net Input (I):** The linear weighted sum plus a bias scalar b.

$$I = \sum_{i=1}^{n} x_i w_i + b$$

- **Output (O):** The result of applying the activation function.

$$O = f(I)$$

- The bias b acts as a threshold to adjust the net input of the unit.

Mathematical Representation of a Unit (Cont.)

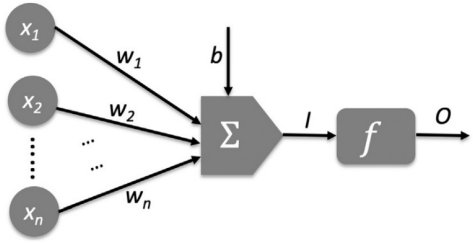


FIGURE 10.2

An illustrative example of unit. Given the inputs $\boldsymbol{X}_1, ..., \boldsymbol{X}_n$, a unit takes a linear weighted sum and then passes the sum through an activation function $f(\cdot)$. $I = \sum_{i=1}^{n} \boldsymbol{X}_i w_i + b$ is the net input of the activation function and $O = f(I)$ is the output of the unit. w_i $(i = 1, ..., n)$ are weights and b is the bias scalar.

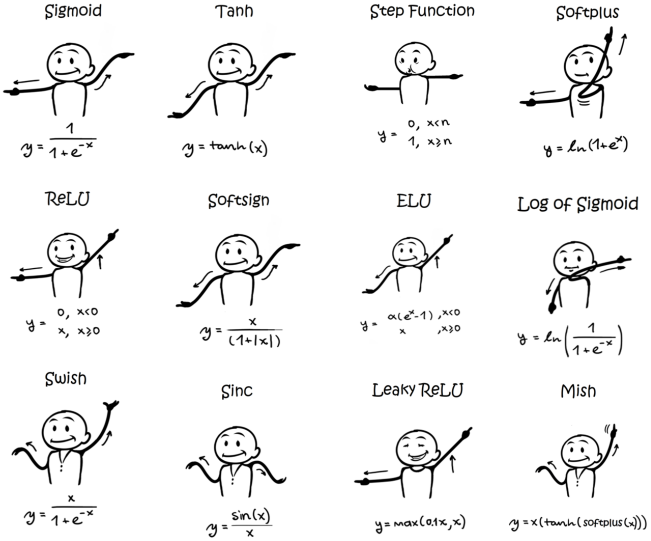
Activation Functions

- Activation functions are typically nonlinear.
- **Step Function:** Used in perceptrons; outputs 1 if $z \geq n$, else 0.
- **Sigmoid Function (σ):** Maps input to (0, 1).

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- **ReLU (Rectified Linear Unit):** $f(I) = I$ if $I > 0$, and 0 otherwise.
- Nonlinearity allows ANNs to model complex, linearly inseparable data.

Activation Functions (Cont.)



Source: The Click Reader (theclickreader.com)

Multilayer Feed-forward Networks

- This is a powerful and common type of ANN architecture.
- Consists of an **input layer**, one or more **hidden layers**, and an **output layer**.
- It is "feed-forward" because weights do not cycle back to previous units or layers.
- It is "fully connected" if each unit provides input to every unit in the next layer.

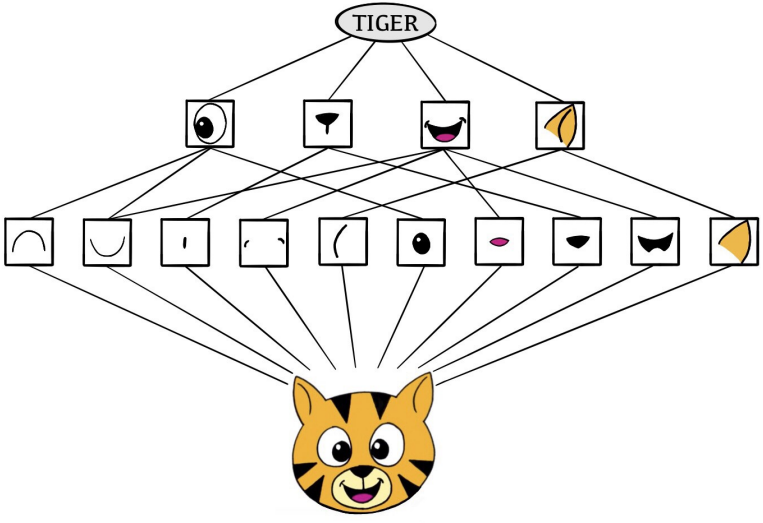
Layer Roles in ANN

- **Input Layer:** Units correspond to the attributes of the input tuple and pass values unchanged.
- **Hidden Layers:** "Neuron-like" units that perform nonlinear transformations.
- **Output Layer:** Emits the final network predictions (e.g., class labels or numeric values).
- A "two-layer" network typically refers to one hidden layer and one output layer (input layer is not counted).

Feature Learning Hierarchy

- Each layer learns features from the outputs of the previous layer.
- This creates a hierarchy of learned features at increasing levels of complexity.
- Example in Computer Vision:
  - Input: Raw pixels.
  - Layer 1: Edges.
  - Layer 2: Contours/Corners.
  - Layer 3: Object parts (e.g., nose).

Feature Learning Hierarchy (Cont.)



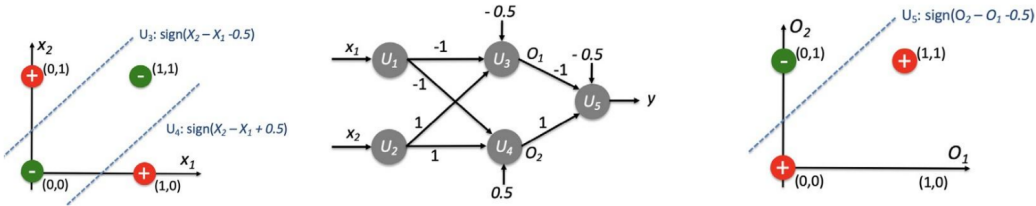
The XOR Problem

- The XOR problem involves four tuples: (0,1) and (1,0) are positive; (0,0) and (1,1) are negative.
- This dataset is **linearly inseparable**.
- A single linear classifier (perceptron) cannot separate the positive and negative tuples.
- This highlights the need for multilayer architectures with nonlinear activation.

Solving XOR with Neural Networks

- A two-layer feed-forward network can solve XOR.
- Two hidden units can act as separate linear classifiers (perceptrons).
- The output unit combines these "learned features" to create a nonlinear decision boundary.
- This transforms the feature space into one where the tuples are linearly separable.

Solving XOR with Neural Networks (Cont.)



(a) Input 4 tuples (b) MLP (c) Outputs of two hidden units

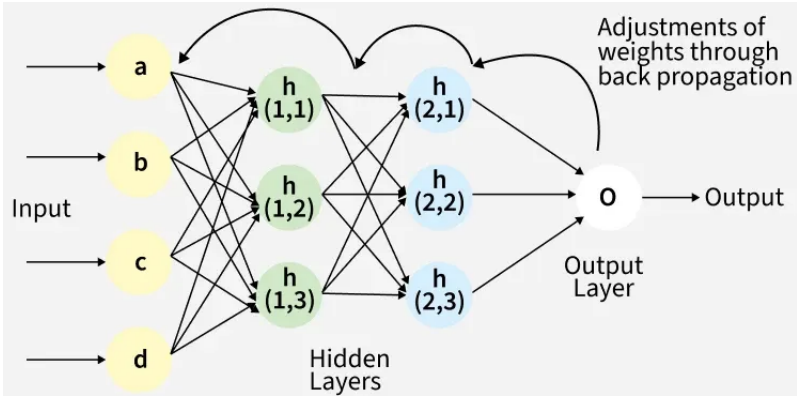
FIGURE 10.3

Solving XOR problem with a two-layer feed-forward neural network with *sign* as the activation function, which is also called multilayer perception (MLP). Note that in (c), both negative tuples share the same representation (O_1, O_2), both being at (0, 1).

The Backpropagation Algorithm

- A technique to automatically learn weights and biases from training data.
- It iteratively processes training tuples and compares predictions to known target values.
- Weights are modified to minimize the loss (e.g., mean-squared error).
- Modifications are made in the "backward" direction, from the output layer to the first hidden layer.

The Backpropagation Algorithm (Cont.)



https://www.geeksforgeeks.org/machine-learning/backpropagation-in-neural-network/

Phase 1: Forward Propagation

- Inputs are fed to the network and pass through the input layer unchanged.
- For each unit in subsequent layers:
  1. Compute net input: $I_j = \sum_i w_{ij} O_i + b_j$.
  2. Compute output: $O_j = f(I_j)$.
- Outputs are calculated layer by layer until the output layer provides a prediction.

Phase 2: Backward Propagation of Error

- The error (δ) is calculated starting from the output layer.
- For an output unit j [1]:

$$\delta_j = O_j(1 - O_j)(T_j - O_j)$$

- For a hidden unit j, error is a weighted sum of the errors of the units in the next higher layer (l):

$$\delta_j = O_j(1 - O_j) \sum_l \delta_l w_{jl}$$

[1] assuming a *sigmoid* activation function.

Phase 3: Weight and Bias Updates

- Parameters are updated using a gradient descent method.
- **Weight Update:** $w_{ij} = w_{ij} - \Delta w_{ij}$

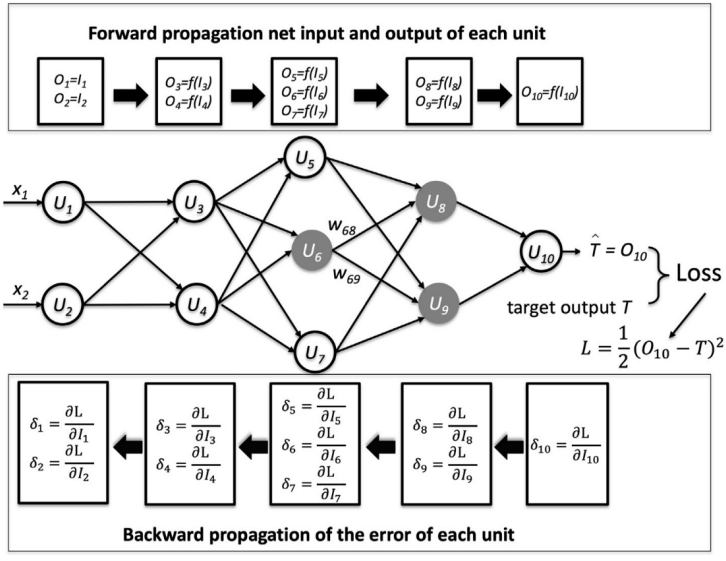
$$\Delta w_{ij} = \eta \delta_j O_i$$

- **Bias Update:** $b_j = b_j - \Delta b_j$

$$\Delta b_j = \eta \delta_j$$

- η is the **learning rate**, usually between 0.0 and 1.0.

The Backpropagation Algorithm (Cont.)



Phase 1: Forward Propagation (Making a Guess)

Passing Clues Forward from the Front Row to the Back

The Classroom Game Setup

Imagine kids sitting in rows (**layers**) playing a guessing game: *"Is this picture a cat or a dog?"* The front row sees the picture, whispers clues to the middle rows, and the last kid shouts the team's guess.

- **1. The First Row (Input Layer):** Inputs are fed to the network and pass through unchanged. The front-row kids get raw clues (*pointy ears, whiskers, furry*) and pass them straight back.
- **2. The Middle Rows (Hidden Layers):** Each kid j listens to the row in front of them and does two steps:
  1. **Gather & Weigh the Clues (Net Input):** $I_j = \sum_i w_{ij} O_i + b_j$
     Kid j listens to every friend's whisper (O_i), multiplies it by how much they **trust** that friend (w_{ij}, the **weight**), and adds their own personal hunch (b_j, the **bias**).
  2. **Decide How Loud to Whisper (Output):** $O_j = f(I_j)$
     Using a special squishing rule (f), kid j turns all those clues (I_j) into a neat score between 0 and 1 and whispers it to the next row.
- **3. The Final Guess (Output Layer):** Whispers travel layer by layer until the very last kid shouts the final prediction!

Phase 2: Backward Propagation of Error (Sharing the Blame)

Walking Backward from the Last Kid to See Who Messed Up

- **Starting at the End:** Suppose the picture was a **cat**, but the last kid shouted **dog**! We calculate the mistake (δ, called "delta") starting at the output layer and walking *backward* row by row.
- **1. The Last Kid's Mistake (Output Unit j, using a sigmoid rule):**

$$\delta_j = O_j(1 - O_j)(T_j - O_j)$$

  - $T_j - O_j$: The gap between the **True answer** (T_j) and the **Kid's guess** (O_j).
  - $O_j(1 - O_j)$: Checks how *stubborn or unsure* the kid was. A kid who was unsure (guessing near 0,5) is much easier to correct!
- **2. Passing the Blame to the Middle Rows (Hidden Unit j):**

$$\delta_j = O_j(1 - O_j) \sum_l \delta_l w_{jl}$$

  - Middle kids didn't shout the final answer, so how do we know their mistake?
  - We check the blame of the kids behind them (δ_l) and how strongly kid j was connected to them (w_{jl}). If you whispered loudly to someone who made a huge mistake, you share a big chunk of their blame!

Phase 3: Weight and Bias Updates (Learning for Next Time)

Adjusting Who You Trust Using Gradient Descent

- **Gradient Descent (Stepping Downhill):** Imagine walking downhill blindfolded by feeling the slope with your feet and taking careful steps. The network takes small steps to shrink its mistakes after every round.
- **1. Changing How Much You Trust Friends (Weight Update):**

$$w_{ij} = w_{ij} - \Delta w_{ij} \qquad \text{where} \qquad \Delta w_{ij} = \eta \delta_j O_i$$

  - Δw_{ij} is the nudge we make to kid j's trust in kid i.
  - It depends on three things: how loudly friend i whispered (O_i), how much blame kid j got (δ_j), and the learning rate (η). If friend i said nothing ($O_i = 0$), they get zero blame!
- **2. Fixing Your Own Personal Hunch (Bias Update):**

$$b_j = b_j - \Delta b_j \qquad \text{where} \qquad \Delta b_j = \eta \delta_j$$

  - Because a hunch comes from inside kid j's own head rather than a friend's whisper, the change only depends on kid j's blame (δ_j) and η.
- **3. The Patience Knob (η, Learning Rate between 0,0 and 1,0):** Controls how fast the kids change their minds—too close to 0,0 means they learn too slowly; too close to 1,0 means they overreact and flip-flop every round!

Quick Cheat Sheet: Kid Game vs. Grown-Up Math

Every Symbol Explained in Plain Language

Symbol	Classroom Kid Game Meaning	Grown-Up Math Name
O_i, O_j	How loudly a kid whispers their clue (0 to 1)	**Output** of a unit
w_{ij}	How much kid j trusts kid i's whisper	**Weight** of the connection
b_j	Kid j's own personal hunch	**Bias**
I_j	All the weighted clues and hunch added together	**Net Input**
T_j	The real answer from the teacher (Cat or Dog)	**Target** value
δ_j	How much blame kid j gets for a wrong guess	**Error** signal
η	The "patience knob" (0,0 to 1,0) for changing minds	**Learning Rate**
Δw_{ij}, Δb_j	The exact amount a kid nudges their trust or hunch	**Weight / Bias Update**

The Big Takeaway

Phase 1 (Forward) passes clues forward to make a guess → **Phase 2 (Backward)** passes blame backward to find out who caused the mistake → **Phase 3 (Updates)** nudges everyone's trust and hunches so the team guesses better next time!

Weight Initialization

- Weights and biases are initialized to small random numbers.
- **Breaking Symmetry:** It is critical that initial weights for units in the same layer are different.
- If weights are identical, units will always update in the same way.
- Random initialization ensures units can learn different features.

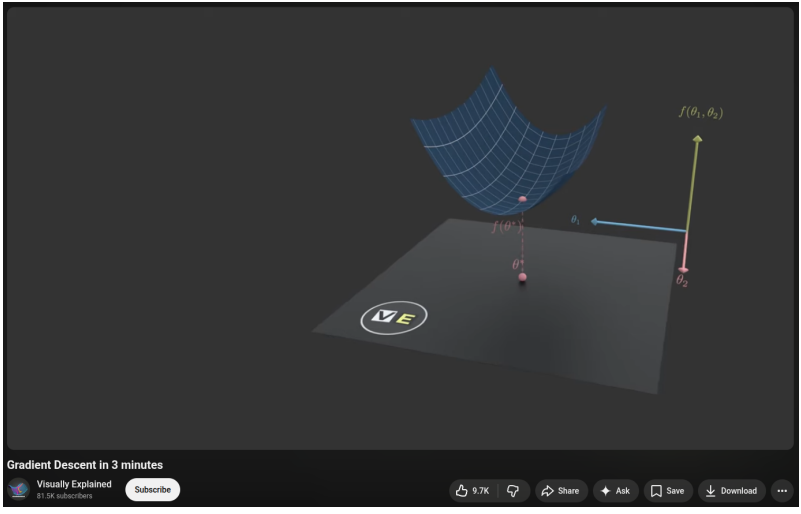
The Learning Rate (η)

- The learning rate determines the size of the steps taken toward optimal weights.
- **Too Small:** Training is very slow.
- **Too Large:** Oscillation between inadequate solutions may occur.
- A common rule of thumb is to set $\eta = 1/t$, where t is the number of iterations.

Gradient Descent vs. Stochastic Gradient Descent

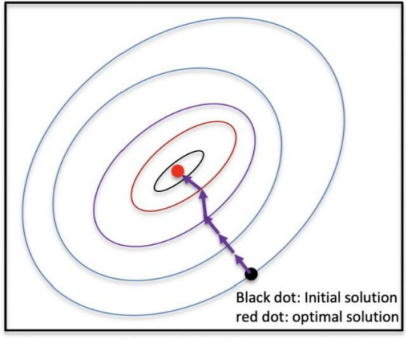
- **Gradient Descent (Full Batch):** Uses all training tuples to compute the gradient.
- **Stochastic Gradient Descent (SGD):** Updates parameters using a small "mini-batch" of tuples.
- SGD finds a direction that approximates the best direction.
- Overall, SGD is often much more computationally efficient.

Gradient Descent vs. Stochastic Gradient Descent (Cont.)

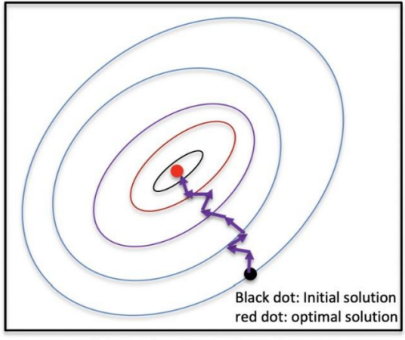


https://www.youtube.com/watch?v=qg4PchTECck

Gradient Descent vs. Stochastic Gradient Descent (Cont.)



(a) Gradient descent



(b) Stochastic gradient descent

Matrix Form Representation

- Neural networks can be represented concisely using matrices.
- A layer transformation can be written as an **affine transformation**:

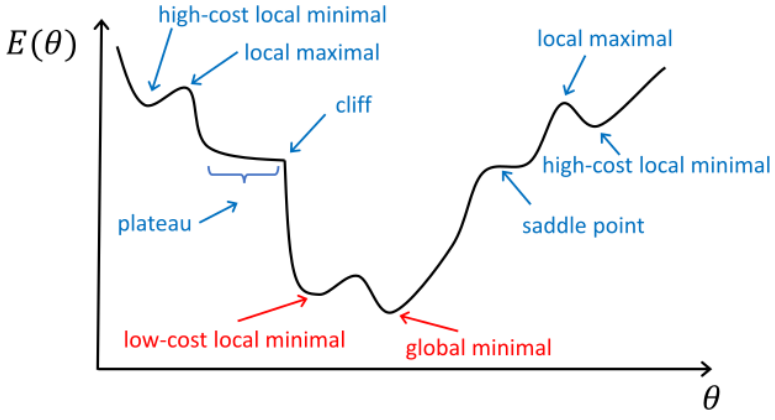
$$h_i = f(W_i h_{i-1} + b_i)$$

- W_i is the weight matrix and b_i is the bias vector.
- Matrix multiplication is often accelerated using GPUs instead of CPUs.

Optimization Challenges

- Training error functions in ANNs are almost always **nonconvex**.
- Potential pitfalls include:
  - **Local Minima:** Ending up in a high-cost area.
  - **Plateaus:** Areas where the gradient is near zero.
  - **Cliffs:** Areas with very large gradients.
  - **Saddle Points:** Gradients are zero but it is not a local minimum.

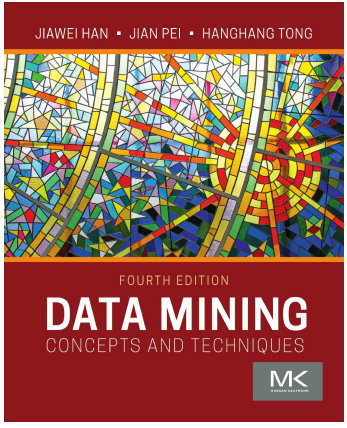
Optimization Challenges (Cont.)



Generalization and Overfitting

- The goal is to minimize the generalization error on unseen test tuples.
- **Overfitting:** Model fits training data perfectly but performs poorly on test data.
- Likely to happen with high model complexity and limited training samples.
- Effective training requires balancing optimization and generalization.

Data Mining: Concepts and Techniques



Content has been extracted from *Data Mining: Concepts and Techniques*, by Jiawei Han & Jian Pei & Hanghang Tong. Morgan Kaufmann. 2022. Visit

https://shop.elsevier.com/books/data-mining/han/978-0-12-811760-6.